Analytics & Big Data

Session 6: Machine learning I

Prof. Dr. Gerit Wagner

(2026-04-20)

Learning objectives

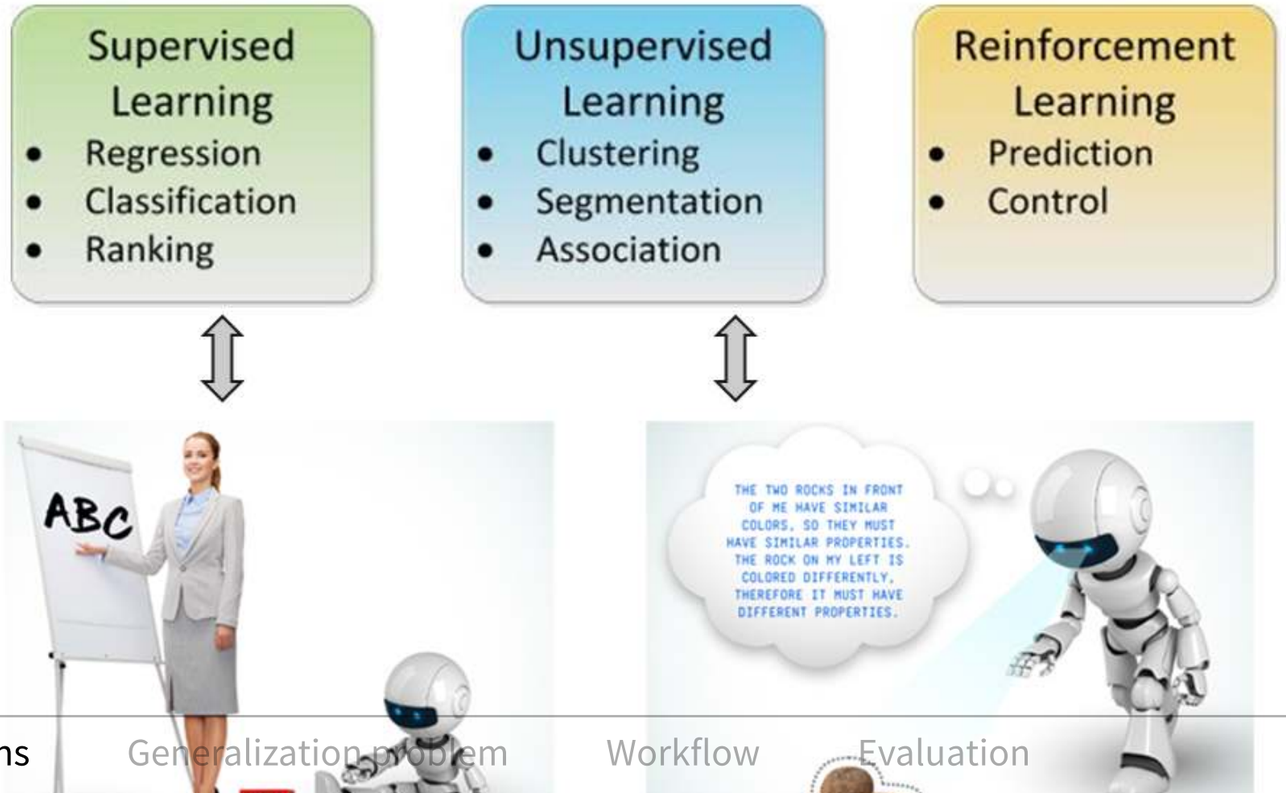
- **Distinguish** between supervised and unsupervised machine learning approaches and explain the generalization problem in supervised machine learning.
- **Describe** the workflow of supervised machine learning, including feature engineering, train–test splitting, model training, cross-validation, and evaluation.
- **Assess** the performance of machine learning models, using the confusion matrix and metrics such as precision, recall and F1 score.
- **Connect** conceptual machine learning procedures to Python implementations, including preprocessing, model training, and evaluation using scikit-learn. (*see exercise*)

Foundations

Today

Categories of ML: Supervised vs. Unsupervised (10 min)

- Define both approaches
- Connect to what students already know:
  - “Unsupervised → clustering” (covered in exploratory data analysis)
- Emphasize: focus of this lecture + next lecture is **supervised learning**



Supervised and Unsupervised Learning

Supervised Learning:

- **Goal: predict a target or outcome variable**
- **Training data where target value is known**
- **Score to test data where value is not known**

DEP_TIME	DEST	DISTANCE	FL_DATE	ORIGIN	Weather	DAY_WEEK	DAY_OF_MONTH	FlightStatus
1455	JFK	184	01.01.2004	BWI	0	4	1	ontime
1640	JFK	213	01.01.2004	DCA	0	4	1	ontime
1245	LGA	229	01.01.2004	IAD	0	4	1	ontime
1709	LGA	229	01.01.2004	IAD	0	4	1	ontime
1035	LGA	229	01.01.2004	IAD	0	4	1	ontime
839	JFK	228	01.01.2004	IAD	0	4	1	ontime
1243	JFK	228	01.01.2004	IAD	0	4	1	ontime
1644	JFK	228	01.01.2004	IAD	0	4	1	ontime
1710	JFK	228	01.01.2004	IAD	0	4	1	ontime
2129	JFK	228	01.01.2004	IAD	0	4	1	ontime
2114	LGA	229	01.01.2004	IAD	0	4	1	ontime
1458	JFK	213	01.01.2004	DCA	0	4	1	ontime
932	LGA	214	01.01.2004	DCA	0	4	1	ontime
1228	LGA	214	01.01.2004	DCA	0	4	1	ontime

HousePrice	HouseSize	HouseAge	LotSize	RiverSide	CrimeRate	Industrial
244	6.064	59.1	0	1	.13587	10.59
212	6.176	72.5	0	0	.13158	10.01
222	6.358	52.9	30	0	.1029	4.93
134	6.103	85.1	0	0	705.042	18.1
238	5.877	79.2	0	0	180.028	19.58
174	5.594	36.8	12.5	0	.13554	6.07
222	6.316	38.1	0	0	.05083	5.19
262	6.718	17.5	22	0	.19073	5.86
140	6.174	93.6	0	0	.2909	21.89
190	5.985	45.4	0	0	.05497	5.19
178	7.393	99.3	0	0	824.809	18.1

Unsupervised Learning:

- **Goal: detect patterns**
- **Grouping of cases based on similarities in input values**
- **There is no target (outcome) variable**

Brief reminder: logistic regression from previous lecture

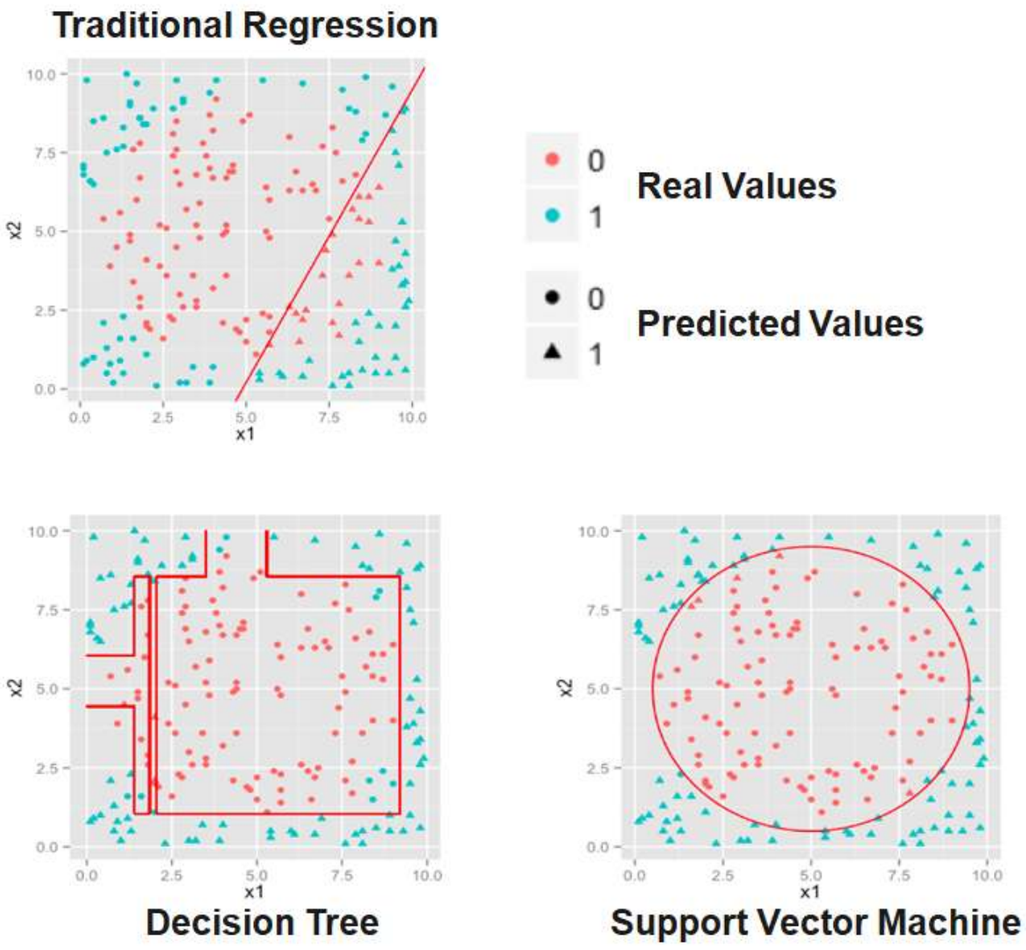
- What problem does it solve?
- Why it is already a form of supervised machine learning

Statistics vs. Machine Learning

Statistics about finding valid conclusions about the underlying applied theory, and on the interpretation of parameters in their models. It insists on proper and rigorous methodology, and is comfortable with making and noting assumptions. It cares about how the data was collected and the resulting properties of the estimator or experiment (e.g. p-value). The focus is on hypothesis testing.

Machine Learning (ML) aims to derive practice-relevant findings from existing data and to apply the trained models to data not previously seen (prediction). It tries to predict or classify with the most accuracy. It cares deeply about scalability and uses the predictions to make decisions. Much of ML is motivated by problems that need to have answers. ML is happy to treat the algorithm as a black box as long as it works.

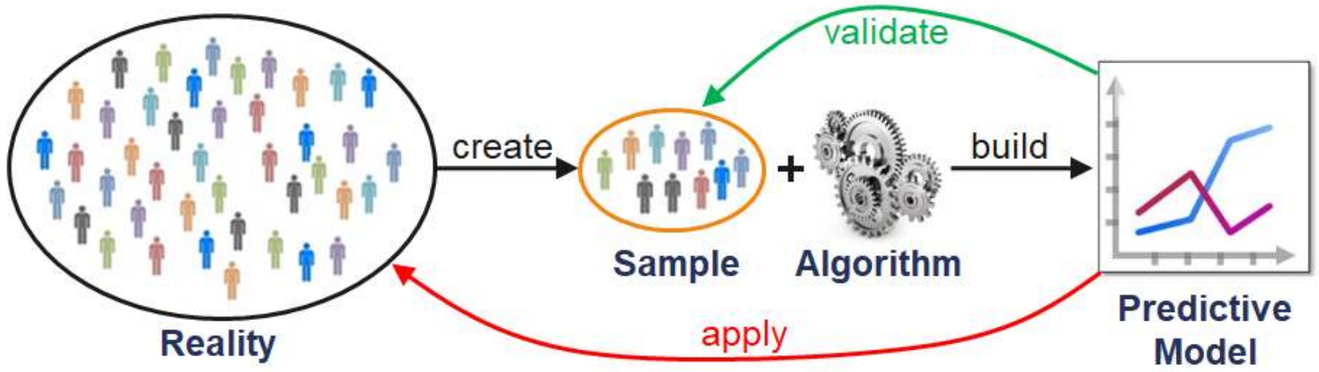
Statistical Regression vs. Machine Learning Algorithms



The Generalization Problem

Overfitting (example, ...)

Traditional Analytics Process



Note: large ML models may create overfitting issues… -> need for separate test set

Generalization problem

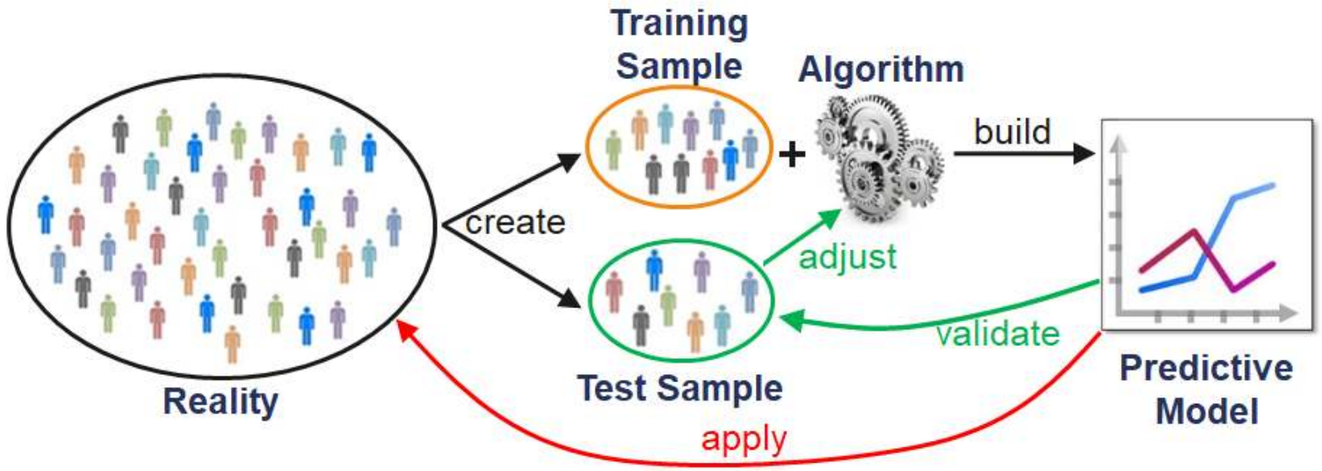
State the generalization problem very clearly.

Ask students for ideas

The Machine Learning Workflow

The Supervised Machine Learning Workflow

Introduce the high-level pipeline (Modern Analytics Process)



2.1 Labeled Data - Input features (X) and target labels (y) - Importance of data quality and representativeness

2.2 Feature Engineering - Transformations, encoding, normalization - Why features matter more than the model itself

2.3 Train–Test Split - Why we split data into training and testing sets - Preventing information leakage - Hold-out, cross-validation (brief mention)

2.4 Model Training - Fit model on training data - Conceptual: “learning patterns/parameters”

2.5 Evaluation on Test Data - Predict on unseen data - Establish generalization performance

Overview of Supervised ML Algorithms

Which Method should I choose?

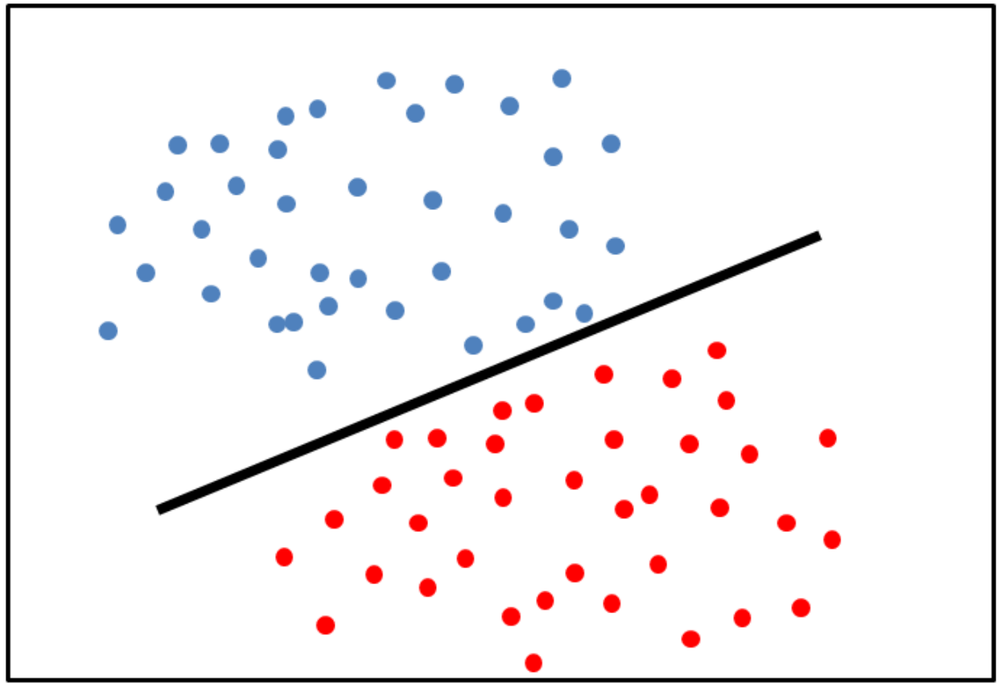
The choice of the method of data analysis depends on the one hand on the scope of application, but on the other hand on the interrelationships of the data to be analyzed.

In the Big Data area, data spaces are often highly-dimensional, making it difficult to visualize the interrelationships.

For this reason, the choice of the method can often not be made ex ante. In these cases, different methods are competitively tried to select the most suitable one.

- **Logistic Regression**
  - Linear boundary, interpretable, limited for non-linear patterns
- **Support Vector Machines (SVM)**
  - Margin maximization
  - Kernel trick for non-linear boundaries
- **Decision Trees**
  - Recursive splitting, intuitive but prone to overfitting
- **Random Forests / Ensembles** (optional brief preview)
  - Reduce overfitting by combining trees
- **Neural Networks**
  - Layers transforming data
  - Extremely flexible but data-hungry and less interpretable

Linear World



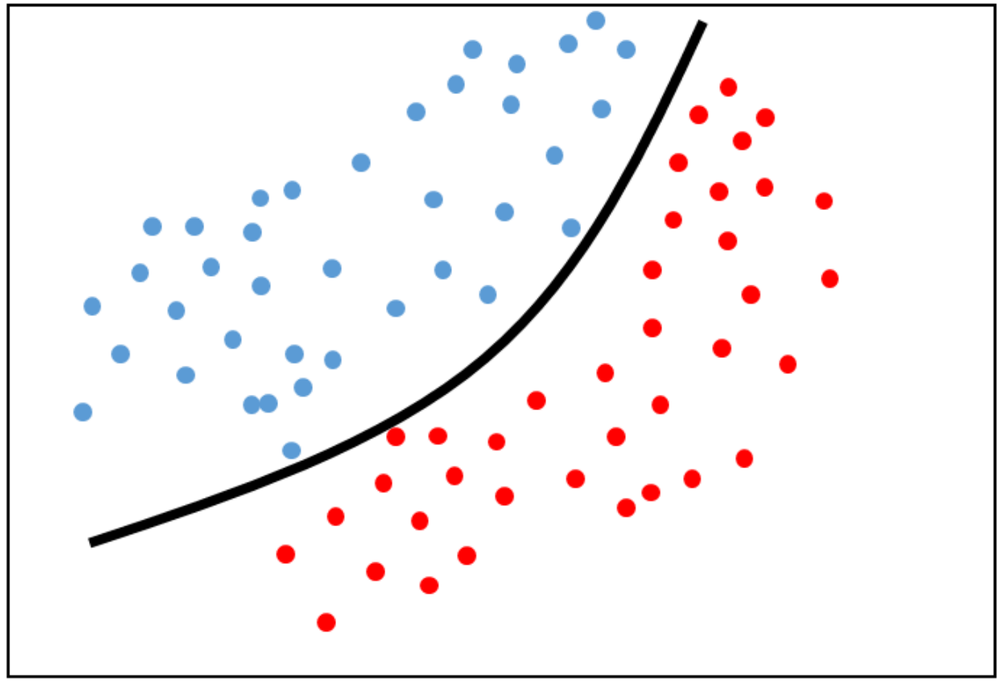
Class 1 e.g. creditworthy

Class 2 e.g. not creditworthy

Appropriate Methods:

- **Regression**
- **Linear Discriminant Analysis**

Quadratic World

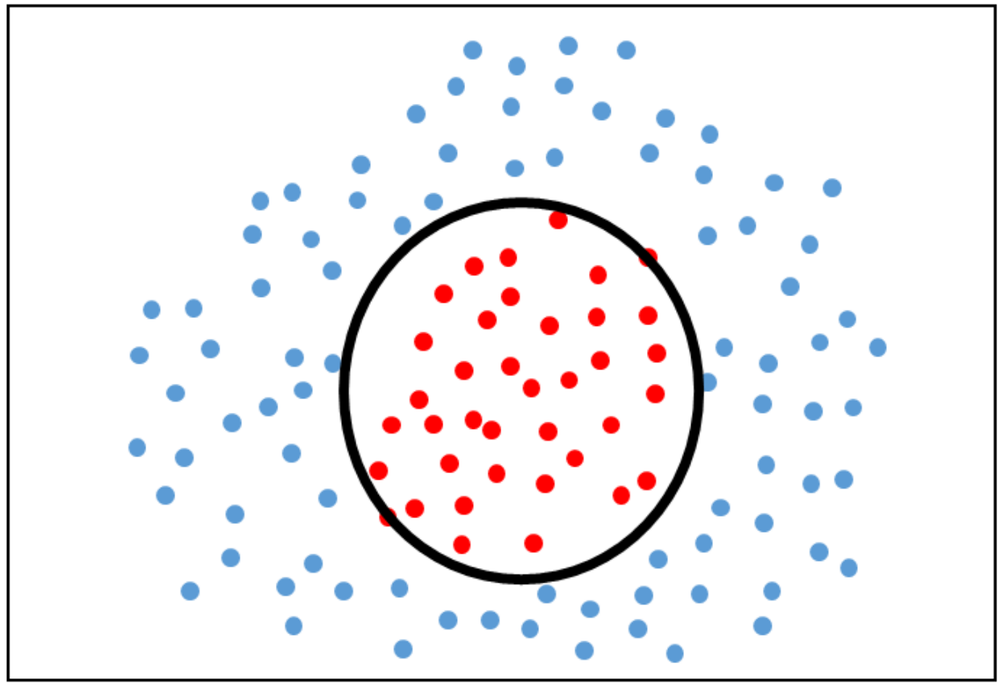


Class 1
Class 2

Appropriate Methods:

- **Quadratic Discriminant Analysis**
- **Support Vector Machines**
- **Neural Networks**

Nonlinear World (Type 1)

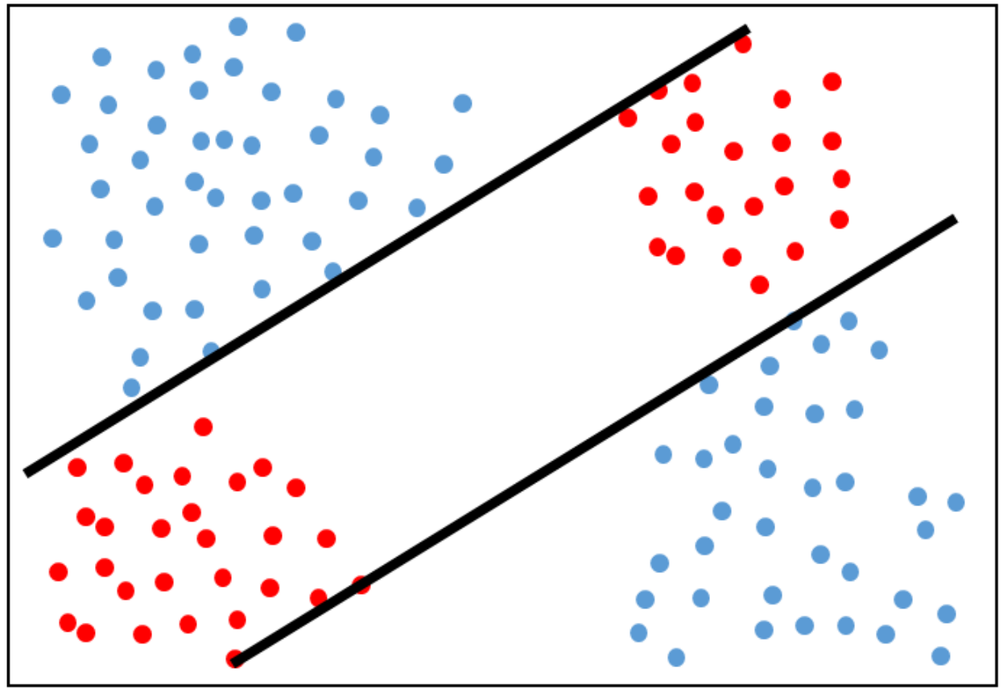


Class 1
Class 2

Appropriate Methods:

- **Support Vector Machines**
- **Neural Networks**

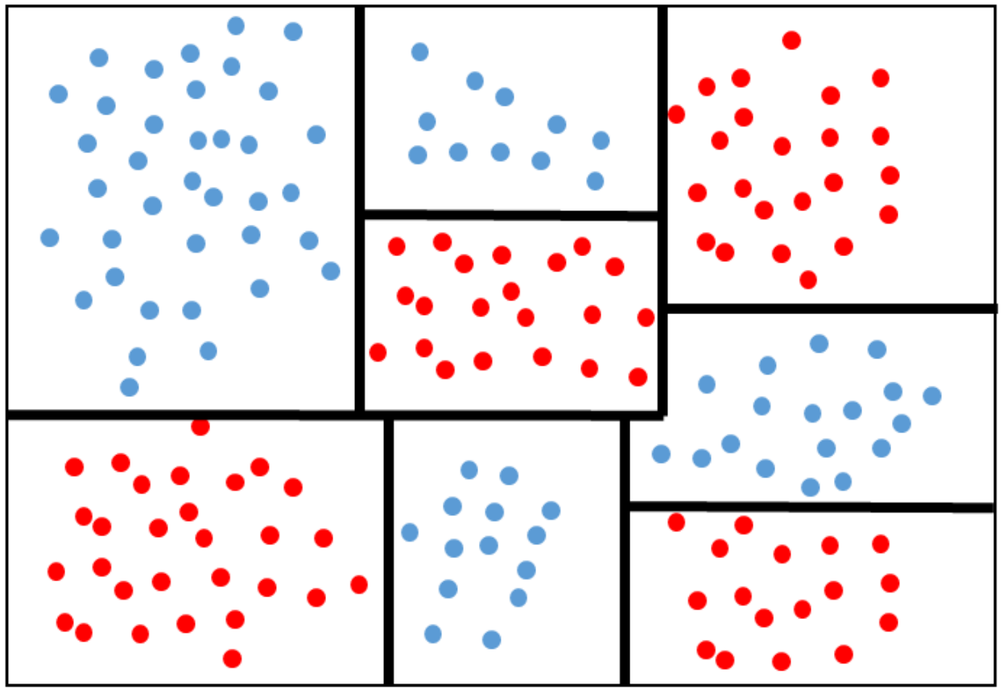
Nonlinear World (Type 2)



Appropriate Methods:

- **Support Vector Machines**
- **Neural Networks**

Nonlinear World (Type 3)

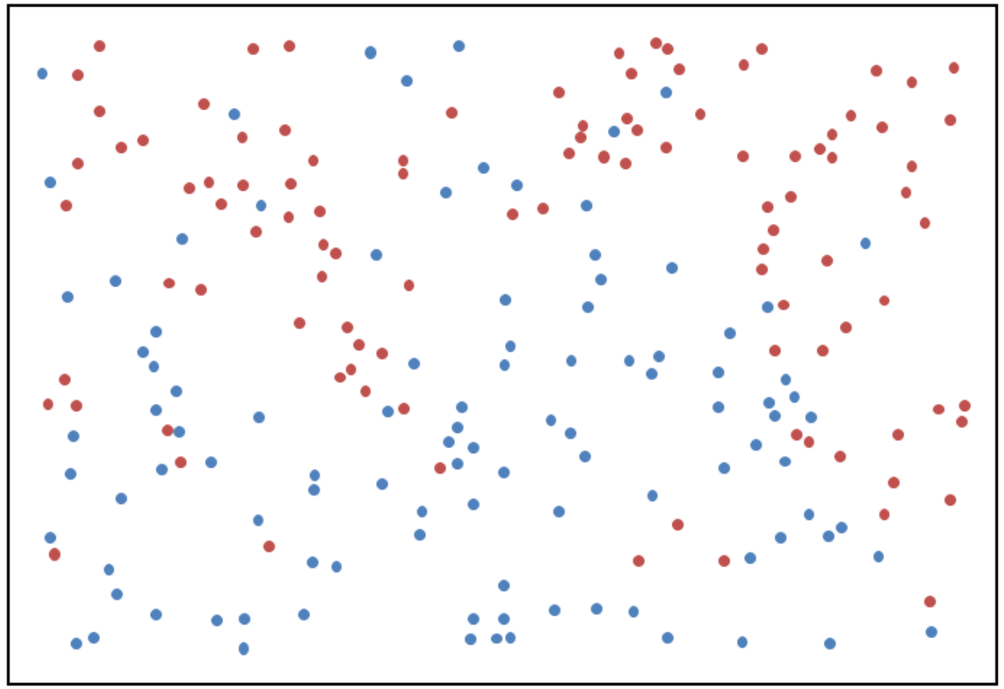


Class 1
Class 2

Appropriate Methods:

- **Decision Trees**
- **Neural Networks**

Nonlinear World (Type 4)



Class 1
Class 2

Appropriate Methods:

- **k-Nearest Neighbors**
- **Naive Bayes**

Feature Engineering

Feature engineering is the process of using domain knowledge of the data to create features that make machine learning algorithms work. If feature engineering is done correctly, it increases the predictive power of machine learning algorithms by creating features from raw data that help facilitate the machine learning process.

A feature (variable, attribute) is depicted by a column in a dataset. Considering a generic two-dimensional dataset, each observation is depicted by a row and each feature by a column, which will have a specific value for an observation:

Columns (Features)

Rows (Observations)

	Feature 1	Feature 2	Feature 3
0	18.38	58.06	-44.21
1	16.10	35.39	77.35
2	23.14	230.44	-71.79
3	24.77	206.83	46.13

Features can be of two major types. **Raw features** are obtained directly from the dataset with no extra data manipulation or engineering. **Derived features** are usually obtained from feature engineering, where we extract features from existing data attributes. A simple example would be creating a new feature “Age” from an employee dataset containing “Birthdate”.

— Source: Sarkar, D.: Understanding Feature Engineering, towardsdatascience.com and Shekhar, A.: What Is Feature Engineering for Machine Learning?, medium.com.

Variants of Feature Engineering

1. Transformation

- convert features (e.g., birth date → age)
- build lag structures (e.g., time-lags)
- normalization / standardization / scaling

2. Type Conversion

- if numerical type is needed, transform categorical into numerical data using dummy features
- if categorical type is needed or more informative, discretize numerical features (e.g., income → poor / rich classes)

3. Feature Combination

- create interaction features (e.g., school_score = num_schools × median_school with num_schools = number of schools within 5 miles of a property and median_school = median quality score of those schools)
- combine categories (e.g., when there are very few observations or too many dummy features)

4. Feature Composition

- build ratios (e.g., returns from prices)
- Principal Component Analysis (Dimensionality Reduction)

Scaling

Most datasets contain features highly varying in magnitudes, units and range.

Most machine learning algorithms have problems with this because they use distance measures or calculate gradients. The features with high magnitudes will weigh in a lot more in the distance calculations than features with low magnitudes and gradients may end up taking a long time or are not accurately calculable.

To overcome this effect, we scale the features to bring them to the same level of magnitudes. The two most discussed scaling methods are Normalization and Standardization.

Normalization typically means to rescale the values into a range of [0,1]:

$$\hat{x}_i = \frac{x_i - x_{min}}{x_{max} - x_{min}}$$

Standardization typically means to rescale data to have a mean of 0 and a standard deviation of 1 (Z scores):

$$\hat{x}_i = \frac{x_i - \mu}{\sigma}$$

If necessary scaling can be inverted:

$$x_i = \hat{x}_i \cdot \left(x_{max} - x_{min}\right) + x_{min} \quad \textbf{resp.} \quad x_i = \hat{x}_i \cdot \sigma + \mu$$

Type Conversion (Encoding)

Many machine learning algorithms cannot work with categorical data directly. To convert categorical data to numbers, there exist two variants:

Label encoding refers to transforming the word labels into numerical form so that the algorithms can understand how to operate on them. Every categorical value is assigned to one numerical value, e.g. young → 1, middle_age → 2, old → 3. This only works in specific situations where you have somewhat continuous-like data, e.g. if the categorical feature is ordinal.

One hot encoding is a representation of a categorical variable as binary vectors. Every categorical value is assigned to an artificial binary variable. If the corresponding categorical value occurs in a data row the value of its binary replacement is equal to 1 else 0, e.g.

Country
Japan
U.S
India
China
U.S
India

Country_Japan	Country_U.S	Country_India	Country_China
1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1
0	1	0	0
0	0	1	0

It is usual when creating dummy variables to have one less variable than the number of categories present to avoid perfect collinearity (dummy variable trap).

Example of Feature Engineering (I)

Data sets often contain date/time features. These features are rarely useful in their original form because they only contain ongoing values. However, they can be useful for extracting cyclical factors, such as weekly or seasonal effects. Suppose, we are given a data “flight date time vs status”. Then, given the date-time data, we have to predict the status of the flight.

But the status of the flight may depend on the hour of the day, not on the date-time. To analyze this, we will create the new feature “Hour_Of_Day”. Using the “Hour_Of_Day” feature, the machine will learn better as this feature is directly related to the status of the flight.

	Date_Time_Combined	Status
0	2018-02-14 20:40	Delayed
1	2018-02-15 10:30	On Time
2	2018-02-14 07:40	On Time
3	2018-02-15 18:10	Delayed
4	2018-02-14 10:20	On Time

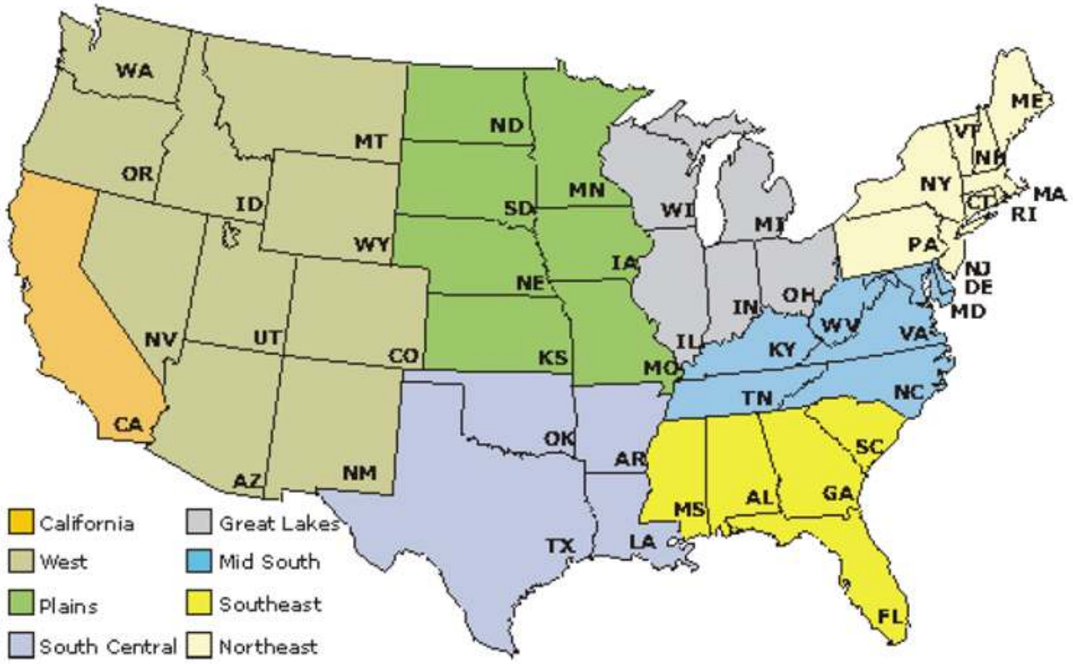
	Hour_Of_Day	Status
0	20	Delayed
1	10	On Time
2	7	On Time
3	18	Delayed
4	10	On Time

— Source: Shekhar, A.: What Is Feature Engineering for Machine Learning?, medium.com.

Example of Feature Engineering (II)

Suppose we are given the latitude, longitude and other data with the objective to predict the target feature “Price_Of_House”. Latitude and longitude are not of use in this context if they are alone. So, we will combine the latitude and the longitude to make one feature.

In other cases, it might be appropriate to transform latitude and longitude into categories which reflect regions, for example.



Example of Feature Engineering (III)

> Suppose we are given a feature “Marital_Status” and other data with the objective to classify customers into “Creditworthy” and “Not_Creditworthy”. In the data set the martial status has many different values, for example:

- single living alone
- single living with his parents
- married living together
- married living separately
- divorced
- divorced but living together
- registered partnerships
- living in marriage-like community
- widowed
- …

> To avoid a transformation into too many and maybe dominating dummy features, we can group the similar classes, e.g. in single, married, widowed.

> If there exist some remaining sparse classes which cannot be assigned in a meaningful way they can be joined into a single

Partitioning the Data

> The partitioning of the data in **Training and Test Data** has the aim to proof if the analytical results can be generalized. The analysis (e.g. the development of a classifier) is carried out on the basis of training data. Subsequently, the results are applied to the test data. If the results are significantly worse than the training data, the model is not generalizable, which is called overfitting.

> The partitioning of the data in training and test data can be carried out in the following ways:

- By random/stratified/… sampling (problem with the repeatability)
- according to a list
- according to rules (e.g. the first/last 50 records or every twelfth)

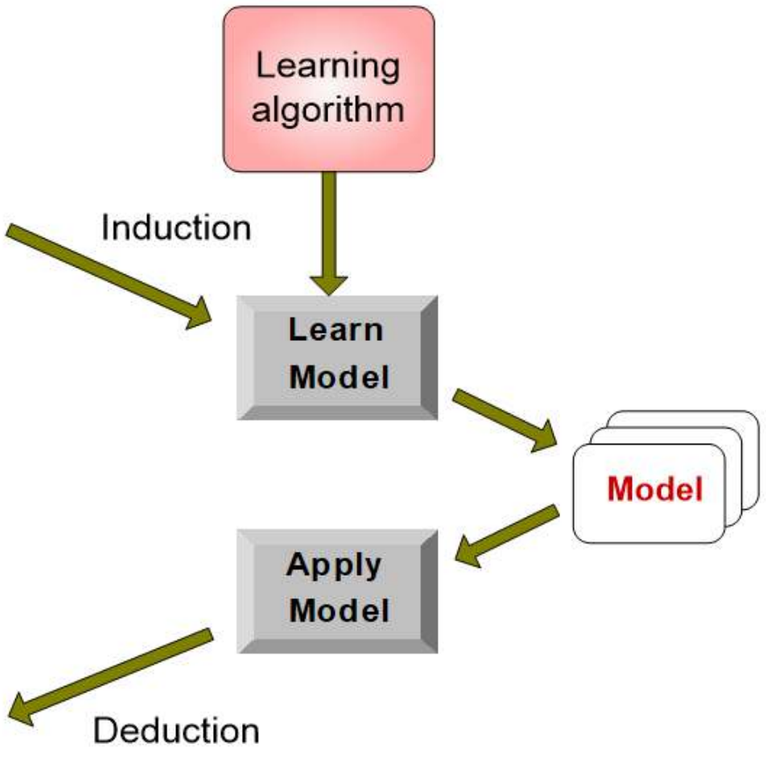
Applying Training and Test Data

Tid	Attrib1	Attrib2	Attrib3	Class
1	Yes	Large	125K	No
2	No	Medium	100K	No
3	No	Small	70K	No
4	Yes	Medium	120K	No
5	No	Large	95K	Yes
6	No	Medium	60K	No
7	Yes	Large	220K	No
8	No	Small	85K	Yes
9	No	Medium	75K	No
10	No	Small	90K	Yes

Training Set

Tid	Attrib1	Attrib2	Attrib3	Class
11	No	Small	55K	?
12	Yes	Medium	80K	?
13	Yes	Large	110K	?
14	No	Small	95K	?
15	No	Large	67K	?

Test Set



— Source: http://www.cs.kent.edu/~jin/BigData/Lecture10-ML-Classification.pptx

Partitioning

Partitions into training and test data

Example:
Training = 60%
Test = 40%

day.mins	day.calls	day.charge	intl.mins	intl.calls	intl.charge	churn
258.6	84	43.96	11.2	5	3.02	No
129.1	137	21.95	12.7	6	3.43	Yes
156.6	88	26.62	12.3	5	3.32	No
332.9	67	56.59	5.4	9	1.46	Yes
190.7	114	32.42	8.1	3	2.19	No
62.4	89	10.61	5.7	6	1.54	Yes
226.1	105	38.44	10.3	5	2.78	No
249.6	118	42.43	11.8	3	3.19	Yes
126.3	102	21.47	9.4	2	2.54	No
173.1	85	29.43	14.6	15	3.94	Yes

Performance

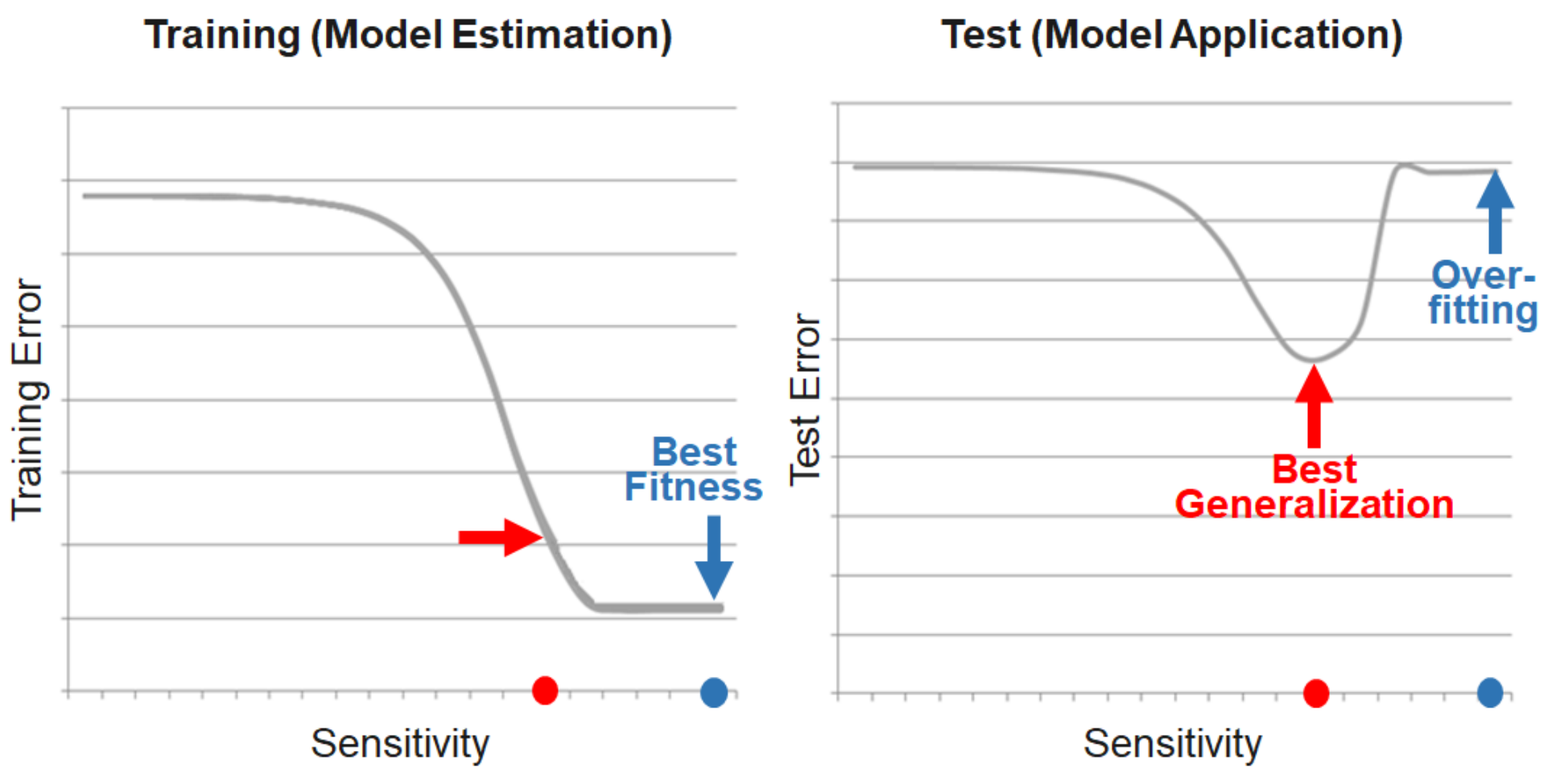
3. Performance Metrics & Model Behavior (15 min)

Introduce classification performance evaluation:

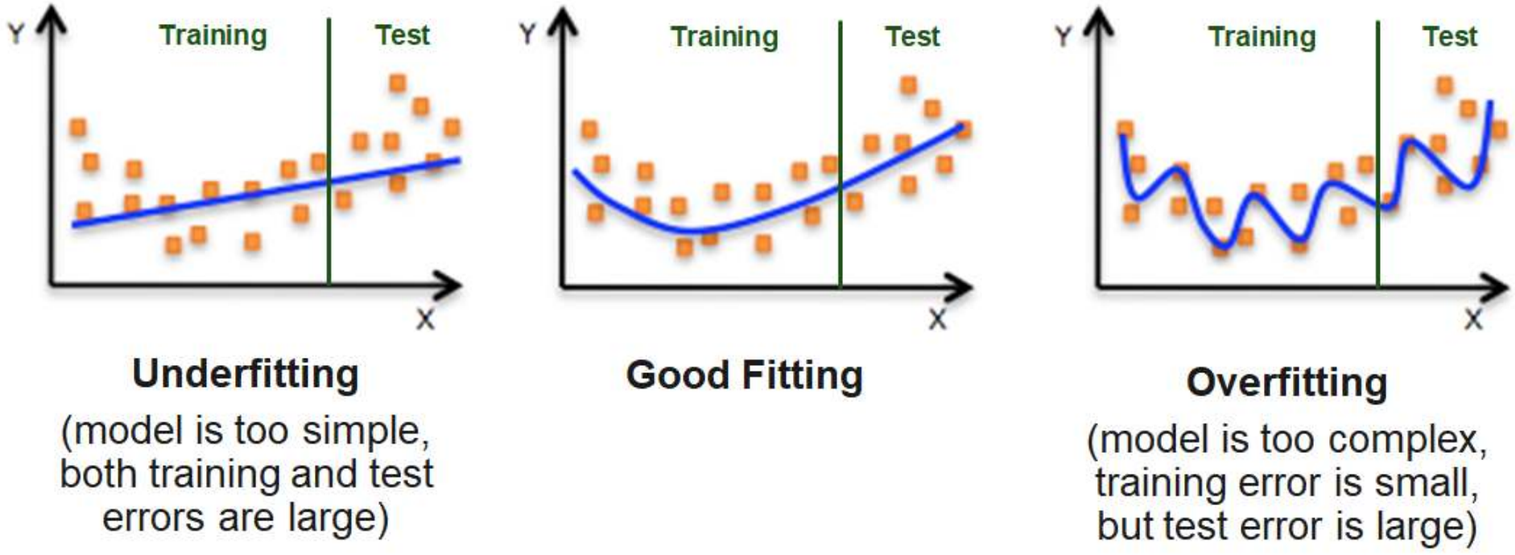
- Confusion matrix (TP, TN, FP, FN)
- Accuracy, Precision, Recall
- F1 Score and when it is useful
- ROC/AUC (optional, light touch)
- Bias–Variance tradeoff (intro)
- Overfitting vs. underfitting
  - High-level intuition: complexity, flexibility, noise-fitting

TODO: cover cases where some error types are more costly than others

Best Fit vs. Best Generalization



Over- and Underfitting



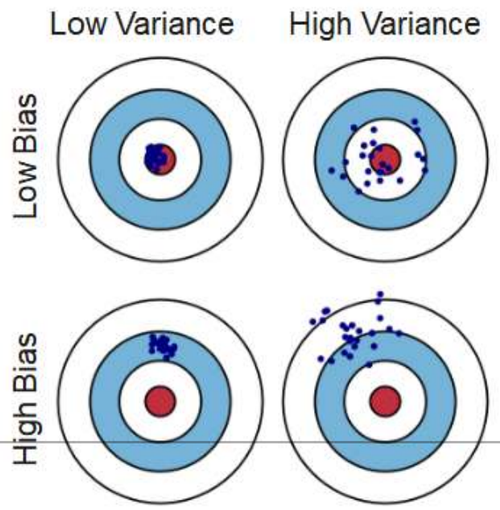
> Due to the problem of overfitting, the main goal is to maximize the prediction quality and not to fit the data that is used for the model estimation as well as possible. This is equivalent to minimizing the risk that the model will have weak predictive ability.

The Bias-Variance Tradeoff

The prediction error is influenced by three components:

> Error = Bias + Variance + Noise

- Bias is the inability of the used method to learn the relevant relations between the inputs and the outputs. It reflects the method quality, e.g. if a method only produces linear models.
- Variance represents the deviation resulting from the sensitivity of the created model to small fluctuations in the data.
- Typically, there is a tradeoff between bias and variance.
- Noise is everything that arises from random variations in the data. It cannot be controlled.



Summary